ICS-344: Information Security

DVSA Vulnerability Discovery & Remediation

Lesson #8: Race Conditions in Order-Processing Workflow

Goal & Vulnerability Overview

Demonstrating how concurrent serverless function invocations can lead to state inconsistency and financial fraud.

AFFECTED ARCHITECTURE

Compute & Entry

API Gateway: Entry point for concurrent requests.

Lambda: Distributed handlers for `ORDER-BILLING` and `ORDER-MANAGER`.

State Persistence

DynamoDB: Stores order records in `DVSA-ORDERS-DB`.

Logic Gap: Pipeline lacks mutual exclusion during the billing phase.

THE ROOT CAUSE: TOCTOU

Time-of-Check to Time-of-Use

The vulnerability exists because of a non-atomic operation window:

- **Check:** Billing reads order total (e.g., $38).
- **Interference:** Update function changes items concurrently.
- **Use:** Billing completes payment using the stale total.

$$\text{Charge}_{\text{final}} = f(\text{State}_{\text{stale}})$$

EXPLOITATION METHODOLOGY

1. Setup

Create a fresh order with 1 item. Add shipping details to prepare for billing.

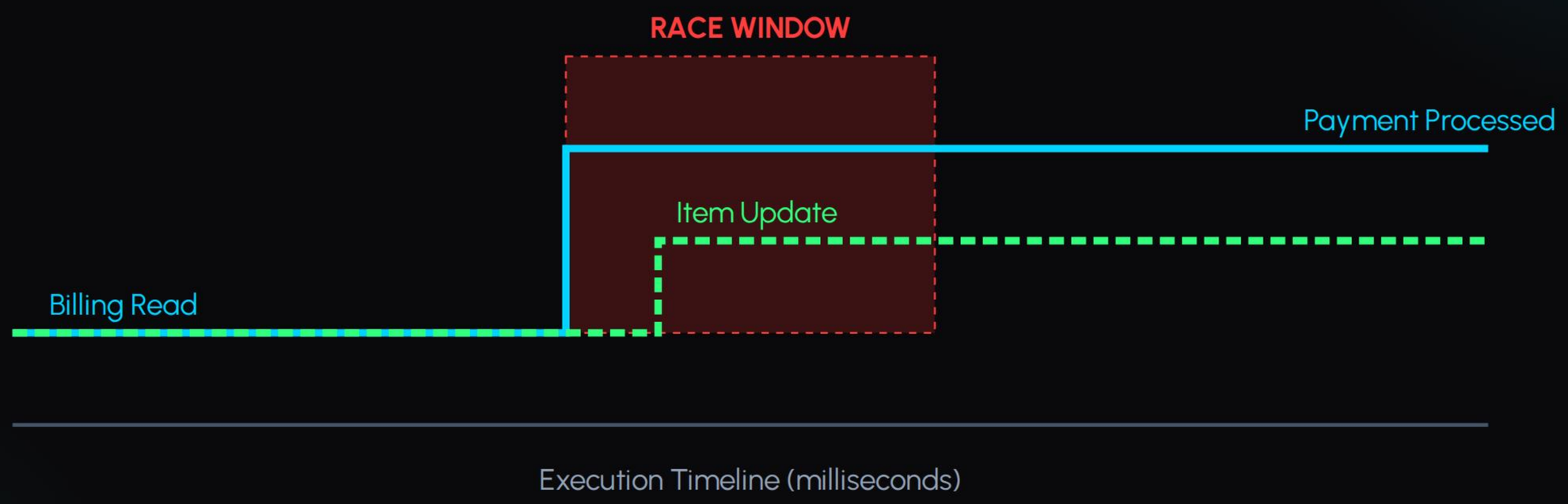
2. The Race

Simultaneously fire Billing and Update requests (using `&` in bash or parallel scripts).

3. Outcome

Pay for 1 item, but the final record shows 5 items delivered. Pure financial fraud.

VISUALIZING THE RACE WINDOW



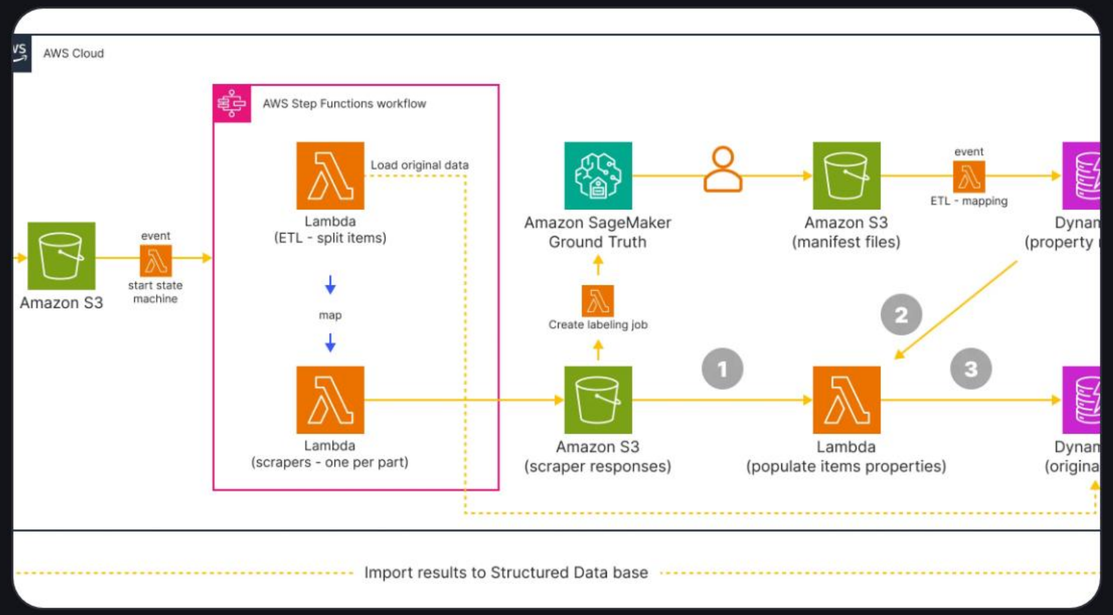
Concurrent processing allows the Update to change the record while Billing is "using" the previous read value.

EVIDENCE: DATA MISMATCH

DynamoDB Attribute	Observed Value	Security Context
`itemList`	`{"1": {"N": "5"}}`	Update Succeeded (Post-Payment)
`orderStatus`	`{"N": "100"}`	Processed (Billing Succeeded)
`totalAmount`	`{"N": "0"}`	Mismatched Charge (Financial Leak)
`paymentTS`	`{"N": "1777482634"}`	Atomic Timestamp confirmed execution

CLOUDWATCH PROOF

Overlapping invocations within milliseconds as seen in AWS Console.



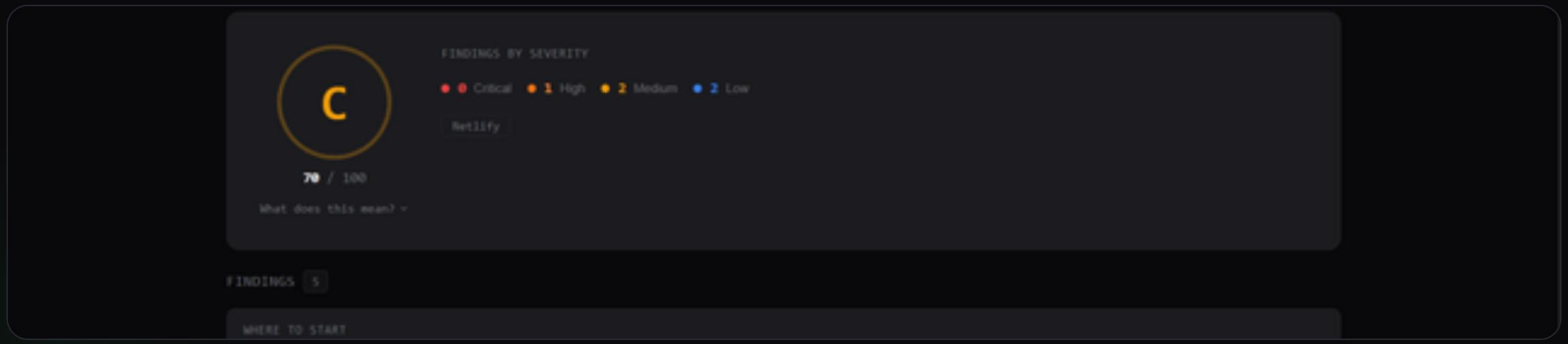
Concurrent execution proof across independent Lambda instances.

Mitigation & Fix Strategy

Implementing atomic state transitions using DynamoDB's Optimistic Concurrency Control.

THE THREE PILLARS OF DEFENSE

- **State Locking:** Update status to "billing-in-progress" atomically upon receiving a request.
- **Conditional Writes:** Utilize `ConditionExpression` to ensure the order is still "open" (100) before any write.
- **Reject Concurrent Updates:** Middleware/Lambda logic must reject item updates if the status is not strictly 100.



CODE REMEDIATION

Vulnerable Write

```
table.update_item(
  Key=key,
  UpdateExpression=expr
)
```

✕ Missing state check.

Atomic Remediation

```
table.update_item(
  Key=key,
  UpdateExpression=expr,
  ConditionExpression="orderStatus = :open"
)
```

✓ Atomic status enforcement.

Q&A

Workflow integrity is a first-class security requirement.

IMAGE SOURCES

https://png.pngtree.com/thumb_back/fh260/background/20250518/pngtree-professional-monitoring-of-network-screens-and-system-data-in-a-cybersecurity-image_17310773.jpg

Source: pngtree.com

---

https://www.ecophon.com/globalassets/new-site/media/images/launches/data-centres/2510_ecophon_data_center_hygiene_black_1_mockup.jpg?width=2560&mode=crop&heightratio=0.5625&quality=90

Source: www.ecophon.com

---

https://images.ctfassets.net/w6r2i5d8q73s/1mdrT2mU09TpfTPZSEr9v2/585f7580282ed25b57d015c9a10ffa18/4.png?fm=webp&q=75

Source: miro.com

---

https://preview.redd.it/45-of-ai-generated-code-has-security-vulnerabilities-i-v0-5hw98wsu8xog1.png?width=640&crop=smart&auto=webp&s=e63f46805ff6509e983580dde5ce2d8edf4c9f7a

Source: www.reddit.com